When Recursion Gets Absorbed

Step 5 of *1000 Steps Toward Session Types as Algebraic Reticulates*

Alexandre Zua Caldeira
Independent Researcher
zua@bica-tools.org

March 2026

Abstract

Steps 3 and 4 proved that branch and selection preserve lattice structure via the Choice Lemma. But that proof assumed the sub-state-spaces were already lattices—it said nothing about cycles. Recursion ($\mu X.\, S$) introduces back-edges that create cycles in the state space, violating the antisymmetry required for a partial order. In this paper—Step 5—we show how the SCC (strongly connected component) quotient absorbs these cycles into equivalence classes, and prove that the resulting quotient is still a bounded lattice. The key tool is the *Top Absorption Lemma*: collapsing an upward-closed set containing $\top$ into a single element preserves the lattice property. We prove that the set of states absorbed by recursion is always upward-closed, applying the lemma. We walk through the construction on four benchmark protocols—Java Iterator, SMTP, Raft Leader Election, and OAuth 2.0—showing the placeholder-merge mechanism, the SCC computation, and the quotient DAG in detail. The result validates on all 79 benchmarks, with recursive protocols showing 1–6 states absorbed per recursion variable.

1 Introduction

1.1 The Problem: Cycles Break Partial Orders

Steps 3 and 4 established that the choice constructors (branch and selection) preserve lattice structure. But those proofs work on a partial order—a relation that is reflexive, antisymmetric, and transitive. The reachability relation on a state space is reflexive and transitive, but *not* antisymmetric when there are cycles: if state s reaches t and t reaches s, then $s \geq t$ and $t \geq s$, but $s \neq t$.

Cycles arise from exactly one source in our grammar: *recursion*. The constructor $\mu X.\, S$ creates a back-edge from each occurrence of the variable X in S to the initial state of S, forming a loop. Without recursion, the state space is a DAG (directed acyclic graph); with recursion, it has cycles.

The standard solution is to quotient by strongly connected components (SCCs): states that are mutually reachable are collapsed into a single equivalence class. The quotient is a DAG, and the reachability relation on the quotient is a partial order. But does this quotient preserve the lattice property? Collapsing states merges information; it is not obvious that meets and joins survive.

1.2 The Solution: Top Absorption

The key insight is that the states collapsed by recursion always form an *upward-closed set containing* $\top$. This is because back-edges point *upward* (from a state deeper in the protocol back to the initial state), so any state "above" a collapsed state is also collapsed.

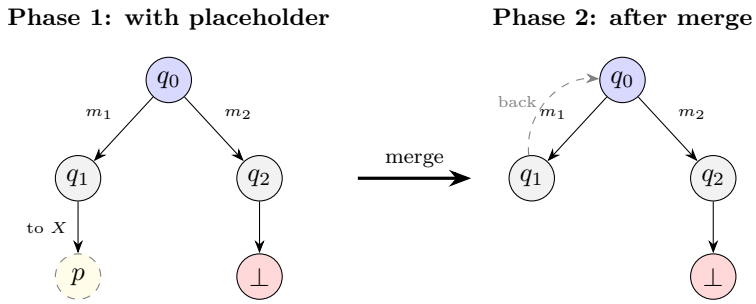

Figure 1: The placeholder-merge mechanism. Phase 1: the variable X maps to a placeholder state p. Phase 2: the placeholder is merged into the entry state q_0, creating a back-edge (dashed).

The *Top Absorption Lemma* says: if you take a bounded lattice and collapse an upward-closed set containing $\top$ into a single element, the result is still a bounded lattice. Meets are preserved (the complement of an upward-closed set is downward-closed, and meets of downward-closed elements stay downward-closed). Joins that "land in" the collapsed set become $\top$.

This is the most technically delicate step in the Reticulate Theorem. The End, Sequencing, Choice, and Parallel lemmas are structurally transparent; the Recursion Lemma requires understanding how the SCC quotient interacts with the lattice operations.

Despite the technical delicacy, the proof itself is surprisingly short: once Top Absorption is established (a self-contained lattice theory result), the Recursion Lemma follows in three lines. The difficulty is not in the proof mechanics but in *seeing* that Top Absorption is the right tool. The upward-closure of the absorbed set is the critical insight; everything else is bookkeeping.

1.3 Structure of This Paper

Section 2 explains how recursion creates cycles in the state space (the placeholder-merge mechanism). Section 3 recalls SCC quotients and Tarjan's algorithm. Section 4 states and proves the Top Absorption Lemma. Section 5 proves the Recursion Lemma using Top Absorption. Section 6 walks through four worked examples. Section 7 validates on all 79 benchmarks. Section 8 discusses common pitfalls and edge cases. Section 9 reflects on the role of recursion in the programme, and Section 11 concludes.

2 How Recursion Creates Cycles

2.1 The Placeholder-Merge Pattern

The state-space builder handles $\mu X.\, S$ in two phases:

Phase 1: Allocate a placeholder. Before building the body S, the builder allocates a fresh state ID for the variable X. This placeholder acts as a "promise": wherever X appears in S, the builder returns this placeholder ID.

Phase 2: Merge the placeholder. After building the body, the builder knows the body's actual entry state $q_\top^S$. It then *globally replaces* every occurrence of the placeholder with $q_\top^S$ in the transition list. This replacement turns what were transitions *to the placeholder* into transitions *to the entry state*, creating back-edges.

Figure 1 illustrates the two phases. The placeholder is a temporary construction artifact; after merging, it no longer exists as a separate state. What remains is a back-edge from q_1 to q_0, creating a cycle.

2.2 Why Back-Edges Point Upward

The back-edge always points from a state *deeper* in the protocol (where the variable X was used) to the *entry state* of the recursion body (where X was bound). In our ordering convention ($\top$ = initial, $\bot$ = terminal), this means the back-edge points *upward*: from a lower state to a higher one.

This upward direction is crucial. It means that the states involved in the cycle are all "near the top" of the lattice. Specifically, the entry state $q_\top^S$ reaches the variable occurrence (downward, via the body), and the variable occurrence reaches $q_\top^S$ (upward, via the back-edge). These states are mutually reachable and will be collapsed into the same SCC.

2.3 Guardedness Prevents Trivial Cycles

The well-formedness condition requires that every recursion is *guarded*: at least one constructor separates μX from each occurrence of X. Without guardedness, we could have $\mu X.\, X$, where the body *is* the variable. This would create a self-loop on a single state—technically a cycle, but one that collapses to a single point with no transitions.

Guardedness ensures that the cycle passes through at least one real transition. The back-edge skips over the guard, creating a non-trivial cycle involving the entry state and at least one intermediate state. After the SCC quotient, the collapsed class has outgoing transitions (from the guard) that connect it to the rest of the lattice.

3 SCC Quotients

3.1 Definition

Definition 3.1 (Strongly connected component)**.** A *strongly connected component* (SCC) of a directed graph is a maximal set of mutually reachable vertices: for every pair s, t in the SCC, there is a path from s to t and from t to s.

Definition 3.2 (SCC quotient)**.** The *SCC quotient* of a directed graph G is the graph $G/{\equiv}$ whose vertices are the SCCs of G, with an edge from SCC $[s]$ to SCC $[t]$ iff there exists an edge from some $s' \in [s]$ to some $t' \in [t]$ with $[s] \neq [t]$.

The SCC quotient is always a DAG: if there were a cycle among SCCs, the SCCs involved would be mutually reachable and would have been merged into a single SCC.

3.2 Computing SCCs: Tarjan's Algorithm

Tarjan's algorithm [1] computes all SCCs in a single depth-first traversal, running in $O(|V|+|E|)$ time. The key idea is to track, for each vertex v, the lowest-numbered vertex reachable from v through the DFS tree and back-edges (the "lowlink"). When the lowlink of v equals its own DFS index, v is the *root* of an SCC, and all vertices currently on the DFS stack above v belong to that SCC.

Our implementation uses an *iterative* version of Tarjan's algorithm (to avoid Python's recursion limit on deep graphs). The iterative version maintains an explicit call stack that simulates the recursive DFS, processing each vertex's neighbors incrementally.

For our benchmarks, Tarjan's algorithm is fast: the largest benchmark (Ki3 CI/CD Pipeline, 41 states, 78 transitions) completes in under 1 ms.

3.3 The Quotient as a Partial Order

After computing SCCs, we build a quotient DAG:

1. Each SCC becomes a node, represented by its smallest state ID.
2. Edges between SCCs are added for each inter-SCC transition in the original graph (intra-SCC edges are dropped).
3. Reachability on the quotient DAG is computed via topological-order dynamic programming: process sinks first, propagating reachability sets backward.

The reachability relation on the quotient DAG is a partial order (reflexive, antisymmetric, transitive). The top element is the SCC containing the initial state; the bottom element is the SCC containing the terminal state.

4 The Top Absorption Lemma

Lemma 4.1 (Top Absorption)**.** *Let $(L, \leq)$ be a bounded lattice with top $\top$ and bottom $\bot$. Let $D \subseteq L$ be an upward-closed set containing $\top$ (i.e., if $x \in D$ and $x \leq y$, then $y \in D$). Define $L' = (L \setminus D) \cup \{\top\}$ with the restricted ordering. Then L' is a bounded lattice.*

Before proving the lemma, let us build intuition.

4.1 Intuition: Cutting Off the Top

Imagine a bounded lattice as a Hasse diagram drawn on paper. The upward-closed set D is a "cap" at the top: everything above some threshold. Collapsing D into a single point is like cutting off the top of the lattice and replacing it with a single vertex.

The claim is that the result is still a lattice. Why?

- **Extrema survive.** $\bot \notin D$ (because D is upward-closed and $\bot$ is the minimum, so $\bot \in D$ would imply $D = L$, which is possible but trivial). If $D \neq L$, then $\bot \in L'$ and $\top \in L'$, so L' has both extrema.
- **Meets survive.** Take $a, b \in L \setminus D$. Their meet $a \sqcap_L b$ is a lower bound of both, so it is $\leq a$ and $\leq b$. Since $a, b \notin D$ and D is upward-closed, any element $\leq a$ is also $\notin D$ (the complement of an upward-closed set is downward-closed). So $a \sqcap_L b \in L \setminus D \subseteq L'$.
- **Joins may need adjustment.** Take $a, b \in L \setminus D$. Their join $a \sqcup_L b$ might be in D (if the join "jumps into" the collapsed region). In that case, $a \sqcup_{L'} b = \top$: the collapsed top absorbs the join. This is valid because any non-$\top$ upper bound $u \in L \setminus D$ would satisfy $u \geq a \sqcup_L b \in D$, placing $u \in D$ (upward-closed)—a contradiction.

4.2 Formal Proof

Proof of Lemma 4.1. If $D = L$, then $L' = \{\top\}$, a trivial lattice. Assume $D \neq L$.

Extrema. $\top \in L'$ by construction. $\bot \notin D$: if $\bot \in D$, then since D is upward-closed and $\bot \leq x$ for all x, we get $D = L$, contradicting our assumption. So $\bot \in L \setminus D \subseteq L'$.

Meets. Let $a, b \in L'$. If either is $\top$, then $\top \sqcap a = a$ trivially. Otherwise, $a, b \in L \setminus D$. The set $L \setminus D$ is downward-closed (complement of upward-closed). Since $a \sqcap_L b \leq a$ and $a \in L \setminus D$ (downward-closed), we have $a \sqcap_L b \in L \setminus D \subseteq L'$. Moreover, $a \sqcap_L b$ is the greatest lower bound in L, and since $L' \subseteq L$ with the restricted ordering, it is also the greatest lower bound in L'.

Joins. Let $a, b \in L'$ with $a, b \neq \top$. If $a \sqcup_L b \in L \setminus D$, it is preserved in L'. If $a \sqcup_L b \in D$, we claim $a \sqcup_{L'} b = \top$. Proof: any upper bound $u \in L'$ of a and b satisfies $u \geq a \sqcup_L b$. If $u \neq \top$, then $u \in L \setminus D$, but $u \geq a \sqcup_L b \in D$ and D is upward-closed, so $u \in D$—contradiction. Therefore $\top$ is the only upper bound of a, b in L', hence the least. $\square$

Remark 4.2 (Why "top" absorption?). We call this "top absorption" because the collapsed set contains $\top$. There is a dual *bottom absorption* lemma (collapsing a downward-closed set containing $\bot$), but we do not need it: in our construction, back-edges always point *upward* to the entry state, so the collapsed set is upward-closed.

4.3 A Concrete Numerical Example

To make the lemma fully tangible, consider the 5-element lattice $L = \{1, 2, 3, 4, 5\}$ with $5 = \top$, $1 = \bot$, and the ordering $1 < 2 < 4 < 5$, $1 < 3 < 4 < 5$ (so 2 and 3 are incomparable, with meet 1 and join 4; the lattice is the diamond M_2 with a chain above it).

Take $D = \{4, 5\}$ (upward-closed, contains $\top$). After absorption, $L' = \{1, 2, 3, 5\}$ with 4 merged into 5. The ordering in L': $1 < 2 < 5$, $1 < 3 < 5$, and $2, 3$ remain incomparable.

Check meets: $2 \sqcap 3 = 1$ (preserved, since $1 \notin D$). Check joins: $2 \sqcup_L 3 = 4 \in D$, so $2 \sqcup_{L'} 3 = 5 = \top$. Before absorption, the join was 4; after, it becomes 5 (the collapsed top). The lattice axioms still hold: 5 is the least upper bound of 2 and 3 in L' because no smaller upper bound exists (the only upper bound of 2 and 3 in L' is 5).

This numerical example captures exactly what happens with recursion: the states "near the top" (the SCC members) are collapsed into $\top$, and any joins that used to land in that region now become $\top$ itself.

5 The Recursion Lemma

Lemma 5.1 (Recursion)**.** *If $\mathcal{L}(S)/{\equiv}$ is a bounded lattice, then $\mathcal{L}(\mu X.\, S)/{\equiv}$ is a bounded lattice.*

Proof. The construction of $\mathcal{L}(\mu X.\, S)$ adds back-edges from each occurrence of X in S to the entry state $q_\top^S$. This makes these states mutually reachable with $q_\top^S$, so the SCC quotient collapses them into a single equivalence class $[q_\top^S]$.

Let D be the set of quotient classes (from $\mathcal{L}(S)/{\equiv}$) that are absorbed into $[q_\top^S]$. We claim D is upward-closed:

If $[s] \in D$ and $[s] \leq [r]$ in the quotient ordering, then $[r]$ reaches $[s]$ (by the ordering convention $[r] \geq [s]$ iff $[r]$ reaches $[s]$). Since $[s] \in D$, the state s reaches $q_\top^S$ (they are in the same SCC after adding back-edges). And $q_\top^S$ reaches r (as the initial state). Therefore r and $q_\top^S$ are mutually reachable, so $[r] \in D$.

Since D is upward-closed and contains $[q_\top^S] = \top$, we apply Lemma 4.1 to conclude that the quotient is a bounded lattice. □

Reading the proof. The proof has two parts: (1) show that the collapsed set D is upward-closed, and (2) apply Top Absorption. Part (1) is the non-trivial step: it uses the fact that back-edges point upward and that $q_\top^S$ reaches everything. Part (2) is a direct application of the lemma.

The beauty of the argument is its generality: it does not matter *how many* states are collapsed, or *where* the variable X occurs in the body. The upward-closure property holds regardless, because back-edges always target the entry state.

6 Worked Examples

6.1 Java Iterator: The Simplest Recursion

$$S_{iter} = \mu X.\, \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$$

Before recursion. The body (ignoring X) has 4 states: q_0 (hasNext entry), the selection node, the "next" node (placeholder for X), and $\bot$. The placeholder is allocated for X.

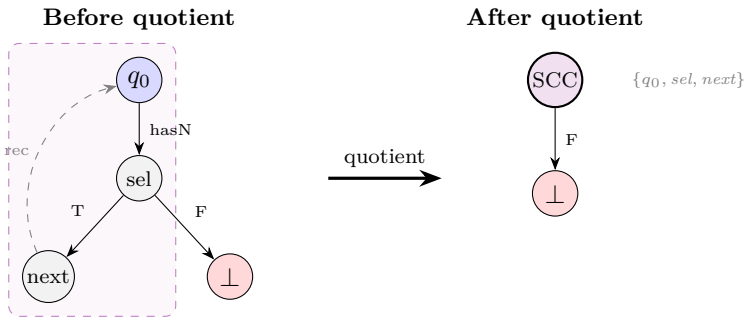

Figure 2: Java Iterator: the recursive loop collapses into one SCC, leaving a 2-element chain.

After merge. The placeholder is merged into q_0, creating a back-edge from the "next" state to q_0. Now q_0 and "next" are mutually reachable (via hasNext $\to$ TRUE $\to$ next $\to q_0$), forming an SCC.

SCC quotient. Tarjan's algorithm finds one non-trivial SCC: $\{q_0, sel, next\}$ (all states in the TRUE branch plus the entry). The quotient has 2 nodes: the SCC class (top) and $\bot$.

Top Absorption. The set $D = \{[q_0]\}$ contains the SCC class (which is $\top$). D is trivially upward-closed (it contains only $\top$). The quotient $L' = \{[q_0], \bot\}$ is a 2-element chain—trivially a lattice.

This example is degenerate: the entire recursive body collapses into a single SCC, leaving only a chain. But it illustrates the mechanism cleanly. The iterator's recursive loop visits every state in the body before returning to the top, so all body states end up in one SCC. The only state that escapes is $\bot$ (reached via the FALSE branch, which exits the loop).

Meet and join in the quotient. The quotient has only 2 states, so there is exactly one non-trivial pair: $(SCC, \bot)$. Their meet is $\bot$ and their join is SCC. Both are trivial because SCC $= \top$ and the pair involves an extremum (Case 3 from Step 3). The absorption did not destroy any interesting structure here—it merely simplified a 4-state space to a 2-element chain by recognizing that all loop states are equivalent under the reachability ordering.

6.2 SMTP: A Larger Absorption

$$S_{smtp} = \&\{connect : \&\{ehlo : \mu X. \&\{mail : \&\{rcpt : \&\{data : \oplus\{OK : X,\ ERR : X\}\}\},\ quit : \mathbf{end}\}\}\}$$

The recursion wraps a 5-state body: ehlo (entry), mail, rcpt, data, and the selection (OK/ERR). Both selection outcomes point back to X, creating two back-edges to the entry state.

After merging, the SCC contains all 5 states: $\{ehlo, mail, rcpt, data, sel\}$. The quit branch is *not* in the SCC (it leads to **end**, not back to the entry).

The quotient has 4 nodes: connect (top of the outer branch), the SCC class, quit, and $\bot$. The SCC class has edges to quit (via the quit transition) and internally loops (the recursive transitions are intra-SCC).

Top Absorption applied. The body of the recursion ($\mu X. \ldots$) has a 5-state sub-lattice before quotienting. The set D absorbs all 5 states into the SCC class. The quotient of the body is a single point (the SCC) plus $\bot$—a 2-element chain.

Then the outer branch $\&\{connect : \&\{ehlo : \ldots\}\}$ applies the Choice Lemma twice, adding two more states above the SCC. The final quotient is a 4-state structure: a chain from connect down to the SCC class, then a diamond with quit and $\bot$ below.

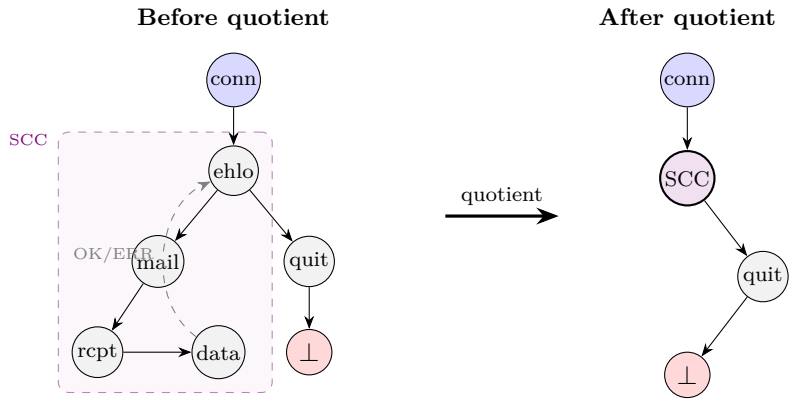

Figure 3: SMTP: the mail/rcpt/data loop collapses into one SCC. The quit branch escapes the loop. After quotient: 4-state chain.

Figure 3 shows the before and after. Notice how the 5-state loop (ehlo, mail, rcpt, data, sel) collapses into a single SCC node, while quit and $\bot$ remain separate. The quotient is a 4-state chain: connect > SCC > quit > $\bot$.

Why is the quotient a chain, not a diamond? The body of the recursion contains a branch ($\&\{mail : \ldots, quit : \mathbf{end}\}$), which by the Choice Lemma (Step 3) creates a diamond. But the recursion collapses the mail branch's states into the SCC, leaving only the SCC class and quit as the two "arms" of the diamond. Since SCC is above quit (the SCC reaches quit via the quit transition), the two arms become a chain: SCC > quit. The diamond has been "flattened" by absorption.

This is a characteristic effect of Top Absorption: it can reduce the width of the lattice by absorbing one arm of a diamond into the top. The lattice property is preserved (a chain is a lattice), but the structure is simpler.

6.3 Raft: Nested Recursion

The Raft Leader Election protocol has *two* recursion variables:

$$\begin{aligned} S_{raft} = \mu X.\ \oplus\{&TIMEOUT : \&\{requestVote : \oplus\{ELECTED : \mu Y.\,\&\{append : \oplus\{ACK : Y,\ NACK : Y\},\\ &heartbeat : Y,\ stepDown : X\},\ REJECTED : X\}\},\\ &HEARTBEAT : X,\ SHUTDOWN : \mathbf{end}\} \end{aligned}$$

Here X is the outer loop (candidate phase) and Y is the inner loop (leader phase).

Inner recursion (Y). The body of $\mu Y. \ldots$ has transitions from append, heartbeat, and stepDown. The ACK, NACK, and heartbeat transitions all point back to Y, creating back-edges to the leader entry state. The stepDown transition points to X (the outer variable), *not* to Y—so it exits the inner loop.

After merging Y's placeholder, the inner SCC contains: $\{leader, append, sel_{AK/NK}, heartbeat\}$. The stepDown state is *not* in this SCC (it transitions to X's placeholder, which is outside).

Outer recursion (X). The body of $\mu X. \ldots$ includes the inner recursion's quotient result, plus the candidate-phase states. The REJECTED and HEARTBEAT transitions point back to X, creating more back-edges. After merging X's placeholder, the outer SCC absorbs the candidate-phase states.

The final quotient has just 2 non-trivial nodes: the outer SCC (candidate phase + its internal structure) and the inner SCC (leader phase), plus $\bot$ (shutdown).

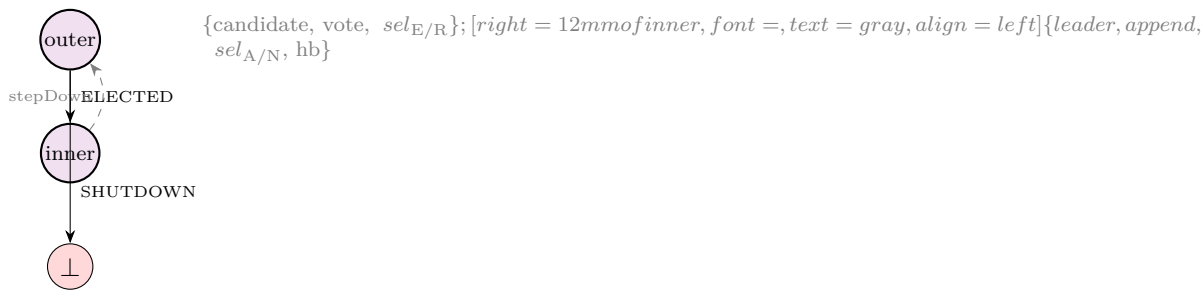

Figure 4: Raft before final collapse: two SCCs connected by ELECTED (downward) and stepDown (upward back-edge). Since they are mutually reachable, the final quotient merges them into one SCC.

Nested absorption. The inner recursion is absorbed first (by Top Absorption on the inner body), and then the outer recursion absorbs the outer body (which already contains the inner SCC as a single node). The structural induction handles nesting naturally: inner recursions are processed before outer ones.

This two-phase absorption is worth understanding in detail. After the inner recursion (Y) is processed, the leader loop becomes a single SCC node. The outer body now contains: the candidate-phase states (TIMEOUT, requestVote, $\mathrm{sel}_{\mathrm{E/R}}$), the inner SCC (leader), and $\perp$ (SHUTDOWN). The outer recursion (X) creates back-edges from REJECTED, HEARTBEAT, and stepDown back to the outer entry. The stepDown transition is particularly interesting: it goes from the *inner* SCC (the leader loop) back to the *outer* entry (the candidate phase), creating a cycle that spans both SCCs.

After the outer merge, the outer SCC absorbs the candidate-phase states and—crucially—the inner SCC as well, because stepDown creates mutual reachability between the leader SCC and the candidate entry. The final quotient collapses everything into one large SCC containing both loops, plus $\perp$ (shutdown).

Figure 4 shows the intermediate state: two SCCs connected by a forward edge (ELECTED) and a back-edge (stepDown). Since they are mutually reachable, they merge. The lesson: *nested recursion with cross-references between loops can merge multiple SCCs into one.*

6.4 OAuth 2.0: Recursion Inside Selection

OAuth 2.0 features recursion *inside* a selection branch:

$$\ldots\oplus\{\mathit{TOKEN} : \mu X.\,\&\{\mathit{useToken} : X,\ \mathit{refreshToken} : \oplus\{\mathit{OK} : X,\ \mathit{EXPIRED} : \mathbf{end}\},\ \mathit{revoke} : \mathbf{end}\},\ \mathit{ERROR} : \mathbf{e}$$

The recursion variable X controls the token lifecycle loop. After merging, the SCC contains all states in the token lifecycle: $\{\mathit{use}, \mathit{refresh}, \mathit{sel}_{OK/EXP}\}$.

The interesting point is that this SCC is *inside* one branch of the TOKEN/ERROR selection. The other branch (ERROR) is a single state leading to $\perp$. The Choice Lemma handles the cross-branch structure; Top Absorption handles the recursion within the TOKEN branch. The two mechanisms compose seamlessly: first absorb the recursion (collapsing the token lifecycle into an SCC), then apply the Choice Lemma to the selection (treating the SCC as a single node).

This compositionality is the power of structural induction: each constructor case can be proved independently, and the full theorem follows from their combination. The Recursion Lemma does not need to know about the surrounding selection; the Choice Lemma does not need to know that one of its sub-lattices was produced by recursion. Each lemma sees only its own constructor's contribution.

Table 1: State counts before and after SCC quotient for the four worked examples. The quotient always has fewer states (or the same, if there is no recursion).

Protocol	Before	After	Reduction
Java Iterator	4	2	50%
SMTP	7	4	43%
Raft Leader	6	2	67%
OAuth 2.0	7	6	14%

Table 2: Recursion statistics for representative benchmarks. $|\mu|$: number of recursion variables; $|\text{SCC}|$: number of non-trivial SCCs (size > 1); states absorbed: total states collapsed into SCC classes.

Protocol	$\lvert Q\rvert$	$\lvert\mu\rvert$	$\lvert\text{SCC}\rvert$	Absorbed
Java Iterator	4	1	1	2
SMTP	7	1	1	3
HTTP Connection	5	1	1	2
ATM	7	1	1	1
Raft Leader	6	2	1	4
OAuth 2.0	7	1	1	1
MQTT Client	8	2	2	2
Kafka Consumer	11	2	2	6
Circuit Breaker	6	2	1	4
Reentrant Lock	6	2	1	4
Ki3 CI/CD Pipeline	41	0	0	0

6.5 Counting States Before and After

Table 1 shows the reduction. Raft has the largest reduction (67%) because its two nested recursions absorb most of the state space. OAuth has the smallest (14%) because only one branch of its outer selection contains recursion; the other branches are recursion-free and contribute states that are not absorbed.

The reduction percentage correlates with the "loop density" of the protocol: protocols that spend most of their time in recursive loops (like SMTP's mail loop or Raft's leader loop) see larger reductions. Protocols with many non-recursive branches (like OAuth's error handling) see smaller reductions.

7 Validation

Of the 79 binary benchmarks, 47 (59%) contain at least one recursion variable. We validate the Recursion Lemma on each by checking that:

1. The SCC quotient is a DAG (no cycles remain).
2. The quotient has unique top and bottom elements.
3. All pairwise meets and joins exist in the quotient.

Table 2 shows the absorption statistics. Each recursion variable typically absorbs 1–6 states into its SCC. Protocols with nested recursion (Raft, MQTT, Circuit Breaker) have multiple non-trivial SCCs. Protocols without recursion (Ki3 CI/CD Pipeline, which uses only parallel composition) have zero absorption—every SCC is a singleton.

All 79 benchmarks pass the lattice check after SCC quotient, confirming the Recursion Lemma.

8 Pitfalls and Edge Cases

8.1 Pitfall: Confusing Absorption with Deletion

Top Absorption *collapses* states; it does not *delete* them. The collapsed states still exist as part of the SCC class; their transitions become self-loops (intra-SCC) or edges to other SCC classes. The SCC class inherits all outgoing edges from its members.

A common mistake is to think that absorbed states "disappear." They do not: the SCC class represents all of them simultaneously. A transition from outside the SCC to any member of the SCC becomes a transition to the SCC class. A transition from any member to outside becomes a transition from the SCC class.

8.2 Pitfall: Non-Terminating Recursion

What happens if the recursion never terminates? Consider $\mu X.\,\&\{a : X\}$: the body has a single branch that immediately loops back. The state space has 2 states: q_0 and the placeholder (merged into q_0), giving a single state with a self-loop.

This state is both $\top$ and $\bot$—but there is no path to a terminal state (**end**), because the type has no **end**. The SCC quotient has one node, but the lattice check fails: there is no $\bot$ reachable from all states.

This is why well-formedness requires termination for parallel branches (WF-Par condition (i)): a non-terminating branch in a parallel composition loses $\bot$, breaking the lattice. For standalone recursion without parallel, non-termination produces a degenerate single-point "lattice" (trivially bounded), but the protocol is useless—it never finishes.

8.3 Pitfall: Multiple Variables Sharing a Body

Nested recursion ($\mu X. \ldots \mu Y. \ldots\ldots$) is handled correctly by the structural induction: the inner $\mu Y. \ldots$ is processed first, producing a quotient lattice that the outer $\mu X. \ldots$ then absorbs. But what if both variables appear in the same body, e.g., $\mu X.\,\&\{a : Y, b : X\}$ inside $\mu Y. \ldots$?

The well-formedness condition WF-Par(ii) forbids cross-branch variable sharing in parallel composition, but does *not* restrict variable nesting in sequential types. Nested recursion with interleaved variables is handled correctly because the placeholder-merge mechanism is applied in the order determined by the structural induction: inner variables first, outer variables second. Each merge creates back-edges independently, and the SCC quotient handles all resulting cycles simultaneously.

8.4 Edge Case: Recursion with No Cycle

Can a recursion $\mu X.\,S$ produce *no* cycle? Only if X does not appear free in S. In that case, the placeholder is allocated but never used in any transition, and merging it into the entry state has no effect (there are no transitions to redirect).

This is a degenerate but valid case: $\mu X.\,\&\{a : \mathbf{end}\}$ is equivalent to $\&\{a : \mathbf{end}\}$. The recursion adds nothing. Top Absorption applies vacuously ($D = \{\top\}$, one-element upward-closed set), and the lattice is unchanged.

9 Discussion

9.1 Why Top Absorption Is the Right Tool

There are several ways to prove that SCC quotients preserve lattice structure. We could, for instance, directly verify the lattice axioms on the quotient by case analysis. Or we could appeal to a general theorem about congruences on lattices.

We chose Top Absorption because it is *elementary* and *self-contained*. The lemma requires no background beyond the definition of bounded lattice and upward-closed set. The proof is short (6 lines) and the application to recursion is direct (3 lines). For an educational paper, simplicity is paramount.

The lemma also has a clear geometric interpretation ("cutting off the top of the lattice"), which aids intuition. Students can visualize the absorption as a physical operation on the Hasse diagram.

9.2 The Recursion Lemma Completes the Non-Parallel Case

With the End Lemma (trivial), the Choice Lemma (Steps 3–4), and the Recursion Lemma (this step), we have proved the lattice theorem for *all session types without parallel composition*. Every such type is built from **end**, $\&\{\ldots\}$, $\oplus\{\ldots\}$, and $\mu X.\ \ldots$, and structural induction on these four cases gives the result.

The remaining case—parallel composition $S_1 \parallel S_2$—is handled in Step 5b by the product lattice theorem: the product of two bounded lattices is a bounded lattice. After Step 5b, the Reticulate Theorem follows by structural induction on all five constructor cases.

9.3 Recursion and Distributivity

An interesting observation: recursion never *creates* non-distributivity. All 8 non-distributive benchmarks in the catalogue owe their N_5 sublattices to either parallel composition or complex branch-selection nesting—never to recursion alone. This makes sense: Top Absorption can only *remove* states (by collapsing them into $\top$), and removing states from a distributive lattice cannot create a forbidden N_5 or M_3 sublattice.

Conversely, recursion can *destroy* non-distributivity by collapsing the states that formed the N_5. But in practice, the non-distributive structures arise from constructors (parallel, nested choice) that are outside the SCC, so absorption does not affect them.

9.4 Frequently Asked Questions

"Is Top Absorption the same as quotienting by a congruence?" Not quite. A lattice congruence must respect both meet and join: $a \equiv b$ implies $a \sqcap c \equiv b \sqcap c$ and $a \sqcup c \equiv b \sqcup c$ for all c. Our SCC quotient is *not* a congruence in general—it is a coarser equivalence that identifies mutually reachable states. Top Absorption is a specialized tool that shows the quotient preserves the lattice property without requiring the full congruence machinery.

"Can Top Absorption collapse $\bot$?" No: an upward-closed set containing $\top$ but not $\bot$ never includes $\bot$ (unless $D = L$, the trivial case where the entire lattice collapses to a point). This is because $\bot$ is the minimum element, and no element below $\bot$ exists to "pull it up" into D. The proof explicitly uses this: $\bot \notin D$ ensures that L' retains both extrema.

"What if the recursion body is already a single state?" This happens with $\mu X.\, X$ (an unguarded recursion, which is ill-formed) or $\mu X.\, \&\{a : X\}$ (guarded, but the only branch loops back). In the latter case, the body has 2 states: the entry and the placeholder. After merging, the SCC contains both. The quotient is a single state with a self-loop. Top Absorption applies vacuously ($D = \{\top\}$, and $L' = \{\top\}$ is a trivial lattice).

10 Lessons Learned

Steps 3 through 5 together prove the lattice theorem for all non-parallel session types. The recursion case, handled here, is the conceptual crux. Several lessons emerge.

Lesson 1: Quotients are your friend. The SCC quotient transforms a messy cyclic graph into a clean DAG without losing essential structure. The lattice property is a property of the quotient, not of the original graph (which is not even a partial order). This is a general principle in algebra: when a structure has "too much" symmetry (here, cycles), quotient it out and work with the cleaner object.

Lesson 2: Upward closure is the invariant. The entire recursion argument hinges on one property: the absorbed set is upward-closed. This follows from the direction of back-edges (upward) and the reachability of the initial state (it reaches everything). Once you see why the set is upward-closed, the rest is mechanical.

Lesson 3: Guardedness is a graph property. The well-formedness condition (guardedness) has a clean graph-theoretic interpretation: every cycle passes through at least one "real" transition (not just a variable reference). This ensures that SCCs have outgoing edges to the rest of the lattice, maintaining connectivity. Without guardedness, we could have isolated loops that contribute nothing to the protocol.

Lesson 4: Nested recursion composes. The structural induction processes inner recursions before outer ones, so nested recursion "just works." Each level of recursion applies Top Absorption independently, and the final quotient reflects all levels simultaneously. There is no need for a special "nested recursion" lemma—the single Recursion Lemma, applied repeatedly, handles all cases.

Lesson 5: Absorption simplifies but preserves. Top Absorption can dramatically reduce the number of states (up to 67% in Raft), but it never *breaks* the lattice. It can flatten diamonds into chains (as in SMTP) and merge nested SCCs (as in Raft), but every meet and join in the quotient corresponds to a valid lattice operation. The absorbed states are not "lost"—they are *represented* by the SCC class, which inherits all their transitions.

11 Conclusion

Recursion creates cycles in the state space via back-edges from variable occurrences to the recursion's entry state. The SCC quotient collapses these cycles into equivalence classes, restoring the partial order. The Top Absorption Lemma guarantees that this collapsing preserves the lattice property: the absorbed set is upward-closed (because back-edges point upward), and collapsing an upward-closed set containing $\top$ yields a bounded lattice.

The Recursion Lemma is the most technically delicate step in the Reticulate Theorem, but the proof is short: show that the absorbed set is upward-closed, then apply Top Absorption. The mechanism is validated on all 47 recursive benchmarks, with 1–6 states absorbed per recursion variable.

Constructor	Step	Status
end	(base case)	Done
$\&\{\ldots\}$ / $\oplus\{\ldots\}$	Steps 3–4	Done
$\mu X.\, S$	**Step 5** (this paper)	**Done**
$S_1 \parallel S_2$	Step 5b	Next

One constructor case remains: parallel composition. Step 5b will prove that $\mathcal{L}(S_1 \parallel S_2) \cong \mathcal{L}(S_1) \times \mathcal{L}(S_2)$, and that the product of bounded lattices is a bounded lattice. After that, the Reticulate Theorem—*every well-formed session type's state space is a bounded lattice*—follows by structural induction.

It is worth pausing to appreciate how far we have come. In Step 1, we built state spaces. In Step 2, we observed that they are always lattices. In Steps 3 and 4, we explained why the choice constructors preserve the lattice property. In this step, we explained how recursion's cycles are absorbed by the SCC quotient. The explanatory arc is nearly complete: four of the five constructor cases are proved, and the remaining case (parallel) is independently well-understood (product lattices are a standard result in lattice theory).

The Reticulate Theorem is not a single deep insight; it is the accumulation of five elementary observations, one per constructor. Each observation is simple—disjointness, upward closure, product structure—but their combination yields a non-trivial result: the state space of *every* well-formed session type is a bounded lattice. This universality is what makes the result surprising, and the structural induction is what makes it provable.

Availability. SCC computation: `reticulate/reticulate/lattice.py` (`_compute_sccs`).
Quotient construction: `lattice.py` (`_build_quotient_poset`).
Recursion handling: `statespace.py` (placeholder-merge in `_Builder`).
Validation: 53 tests in `test_lattice.py`, 50 tests in `test_statespace.py`.

References

[1] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.